

LAPORAN PRAKTIKUM
POSTTEST (7)
ALGORITMA PEMROGRAMAN LANJUT

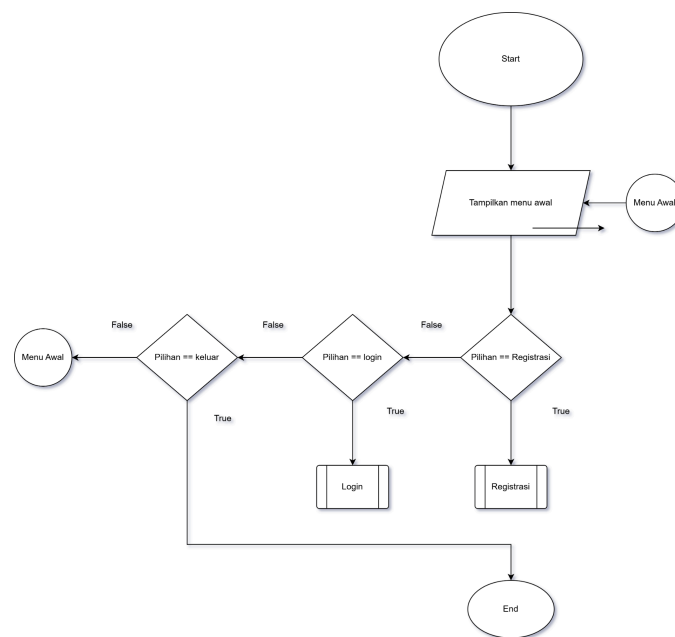


Disusun oleh:
Khayzan Dwitra Attala (2509106103)
KELAS (C'1)

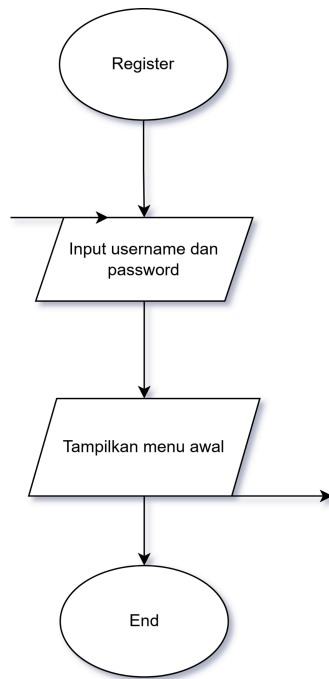
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart

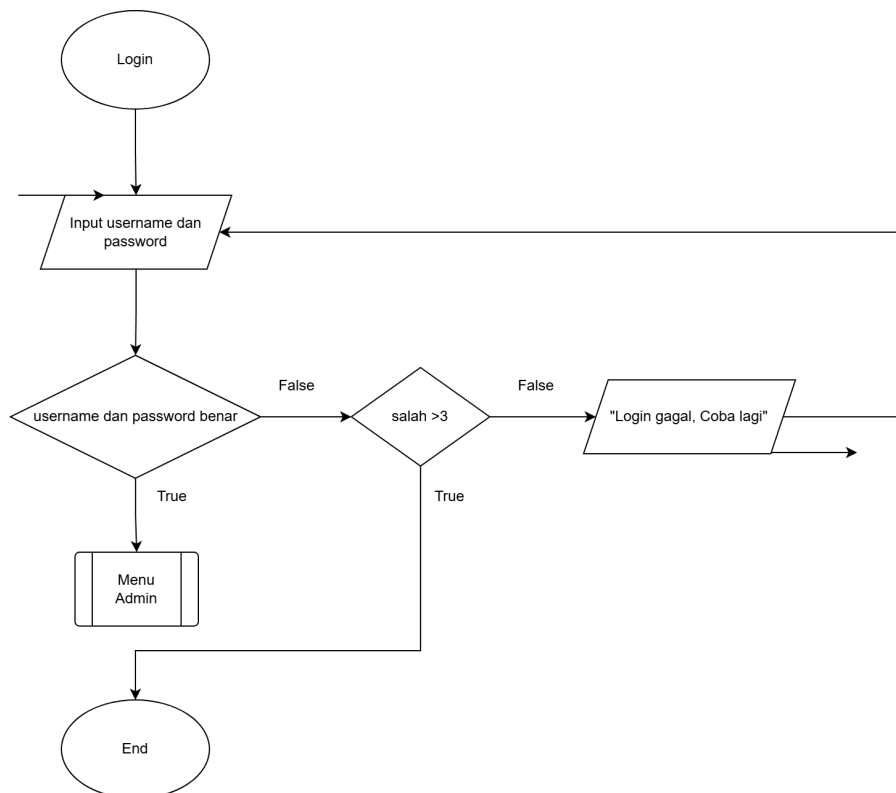
Flowchart ini membantu memvisualisasikan keputusan bercabang (pakai simbol belah ketupat) dan tindakan proses (pakai persegi panjang), sehingga mudah melihat jalur mana yang ditempuh sesuai kondisi.. Kemudian flowchart menggunakan predefined proses agar menunjukkan bahwa ada proses/fungsi yang terdefiniskan



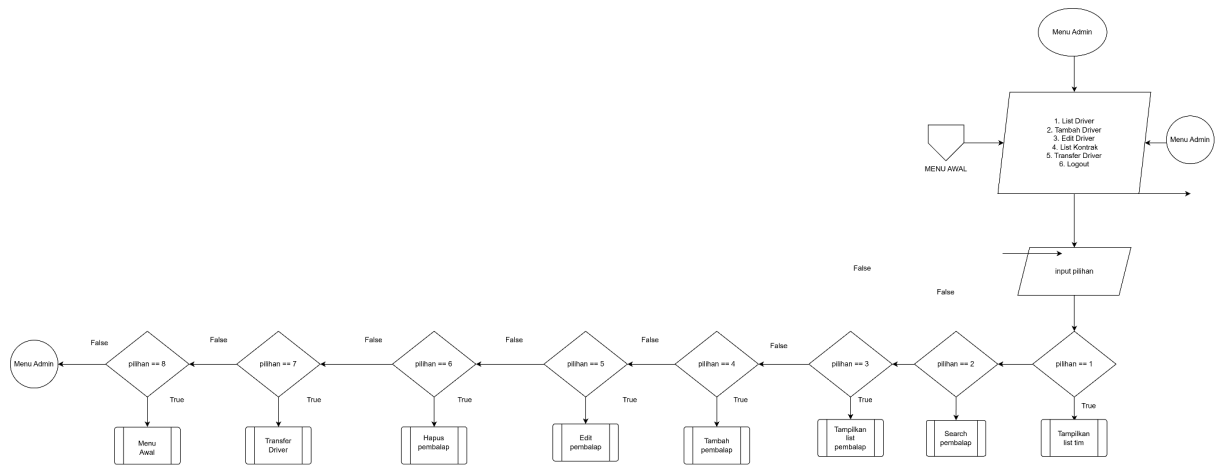
Gambar 1.1 Flowchart CRUD Menu awal



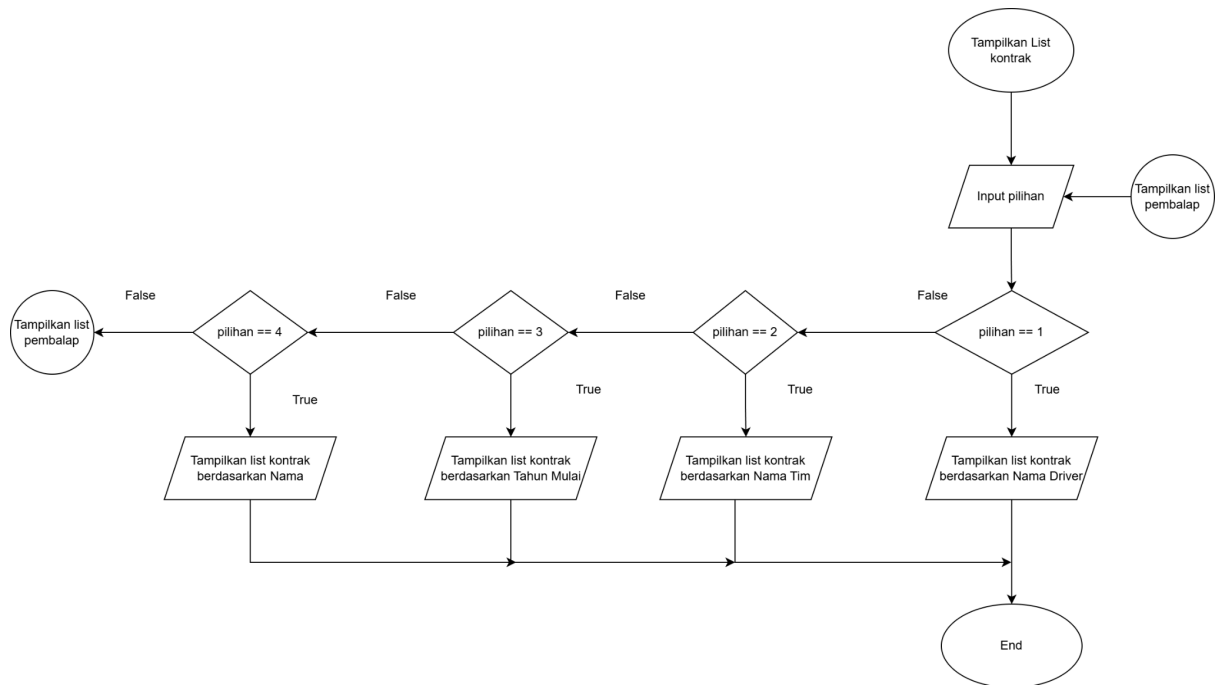
Gambar 1.2 Flowchart Register



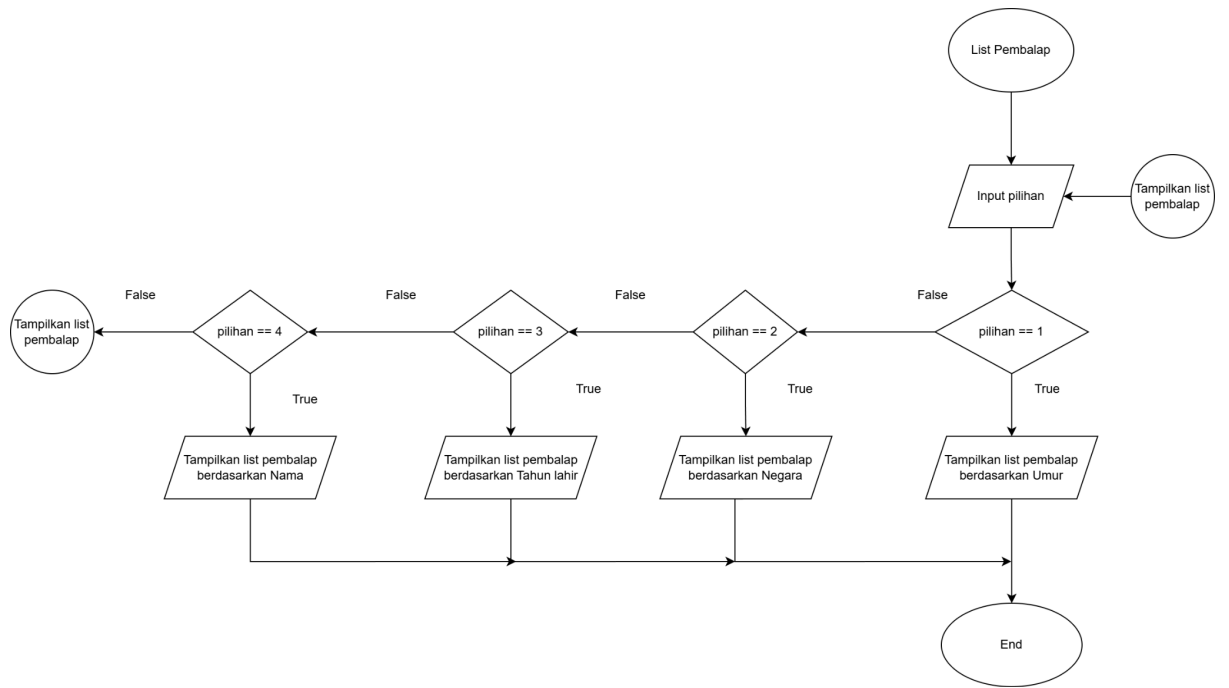
Gambar 1.3 Flowchart Login



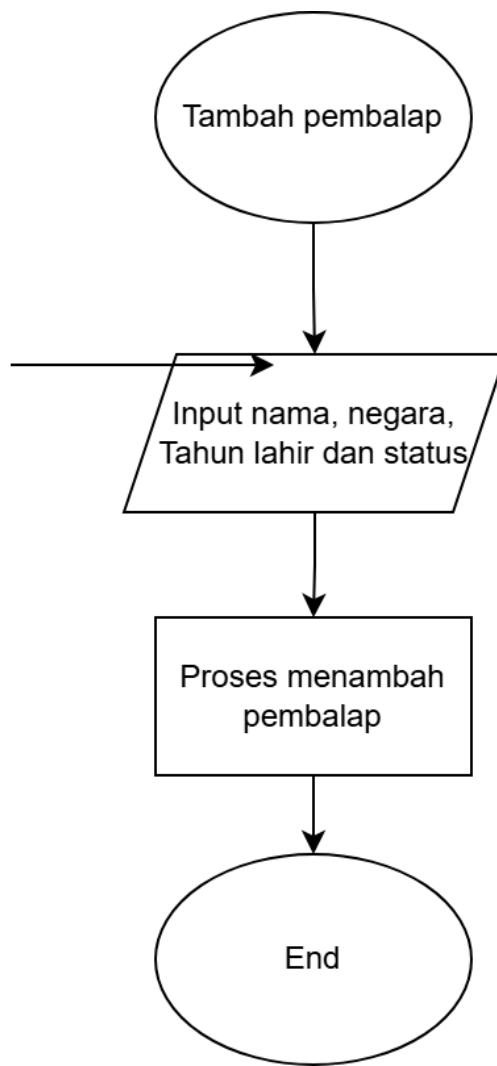
Gambar 1.4 Flowchart Menu Admin



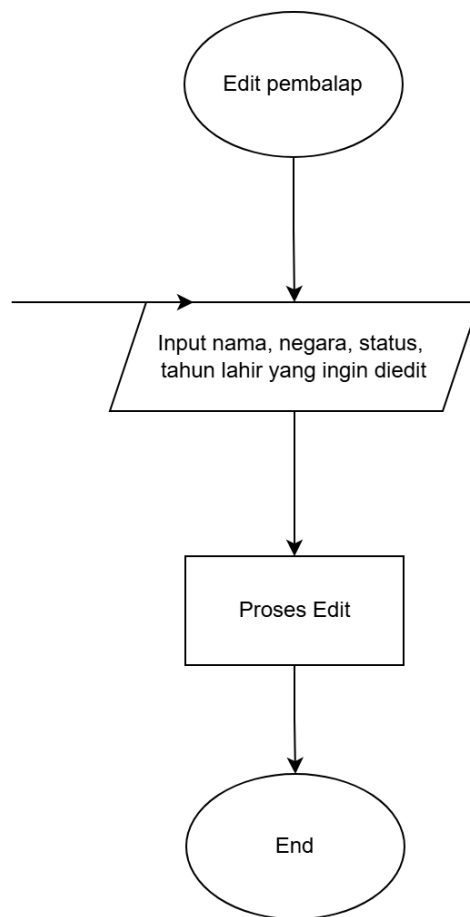
Gambar 1.5 Tampilkan List Kontrak



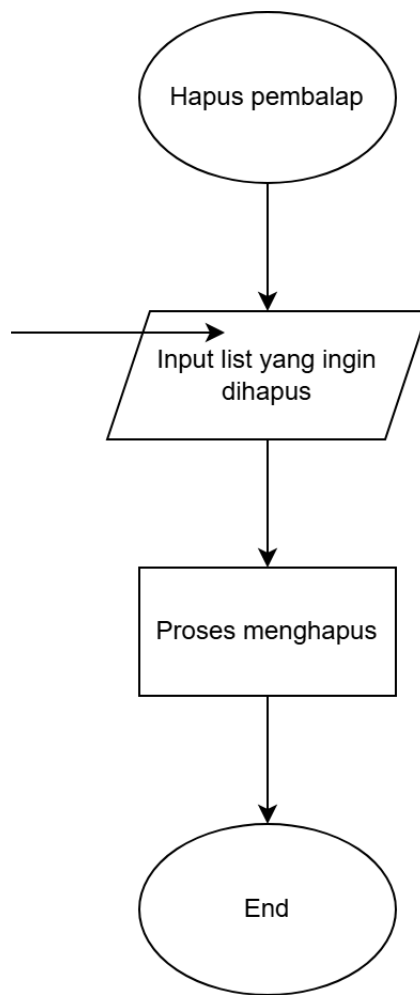
Gambar 1.6 Tampilkan list pembalap



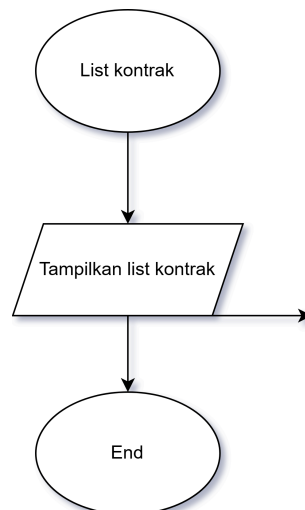
Gambar 1.7 Tambah pembalap



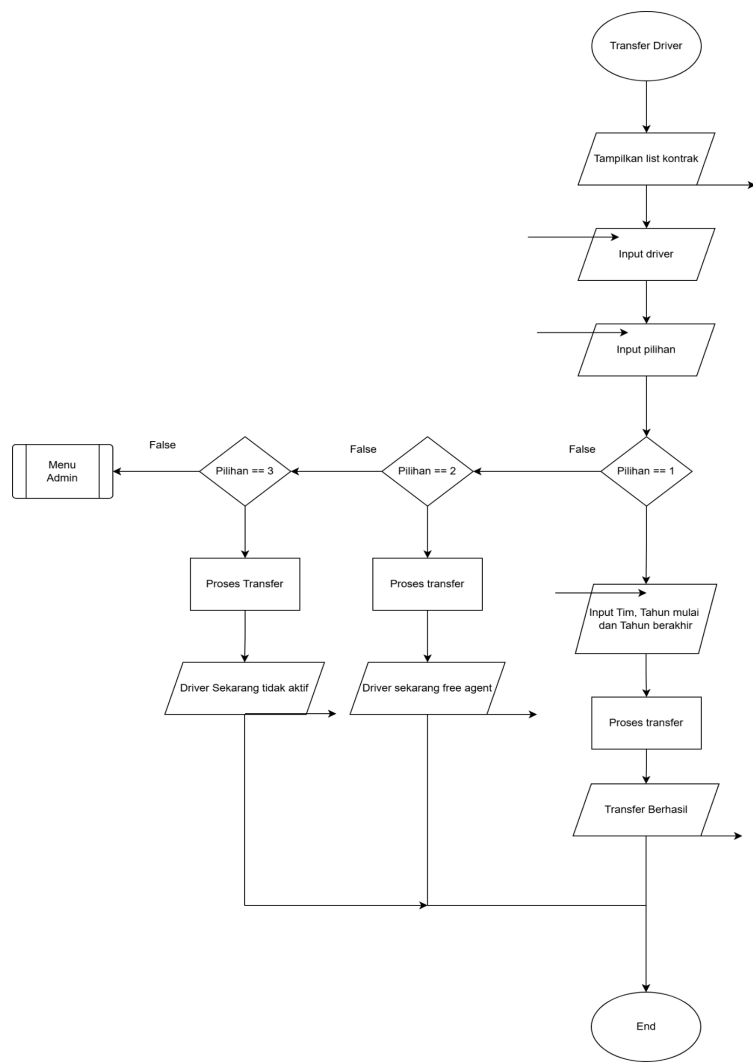
Gambar 1.8 Edit Pembalap



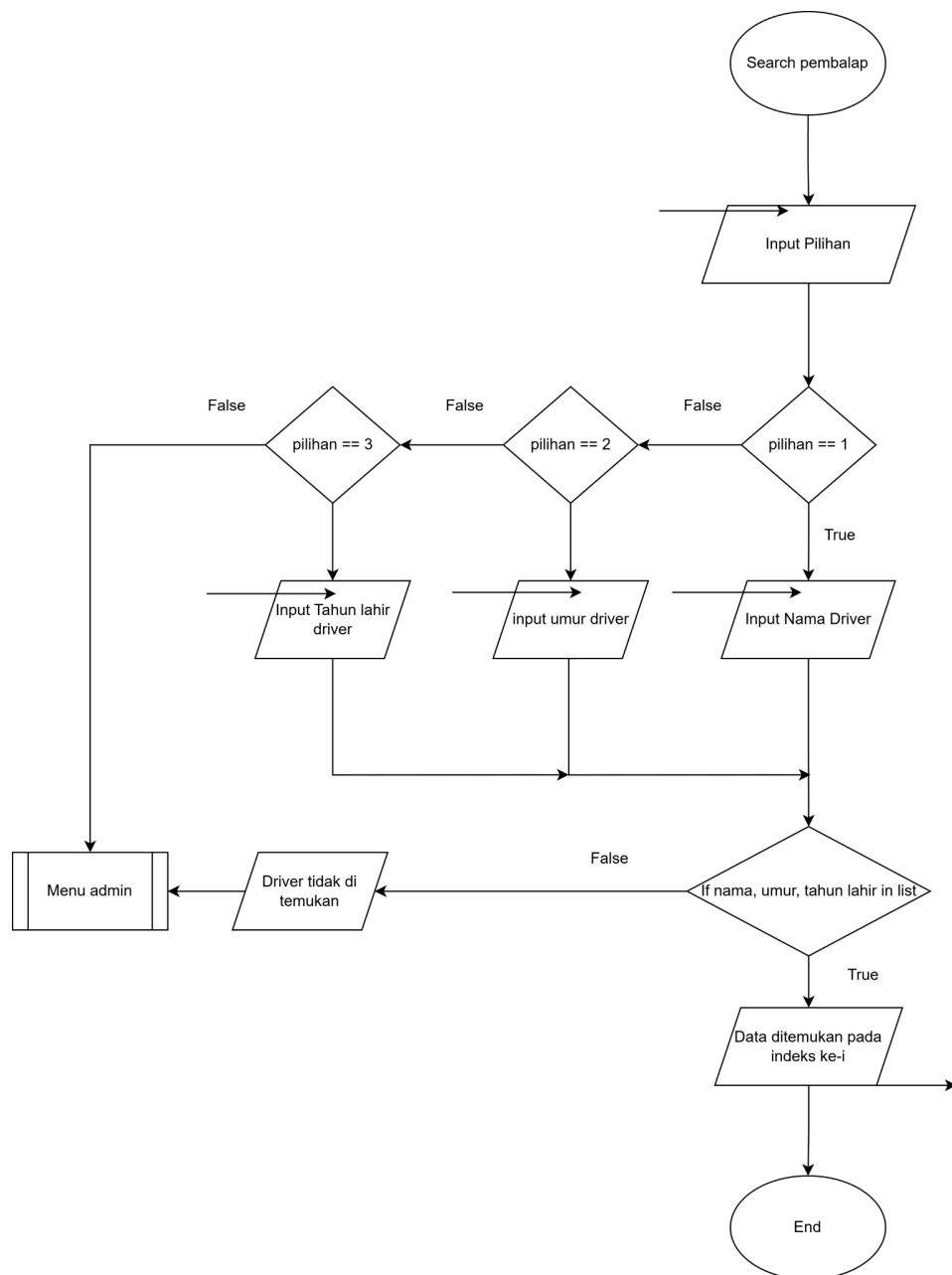
Gambar 1.9 Hapus pembalap



Gambar 1.10 List Kontrak



Gambar 1.11 Transfer Driver



Gambar 1.12 Menu Search Driver

2. Deskripsi Singkat Program

Secara keseluruhan, program ini berfungsi sebagai sistem manajemen kontrak dan informasi pembalap F1 yang sederhana, dengan kontrol akses berdasarkan role dan menu interaktif di konsol.

3. Source Code

```
#include <iostream>
#include <string>
#include <iomanip>
#include <utility>
#include <limits>
#include <cctype>

using namespace std;

const int MAX_DRIVERS = 50;
const int MAX_USERS = 20;
const int MAX_TEAMS = 11;
const int MAX_CONTRACTS = 100;
const int CURRENT_YEAR = 2026;
const int RADIX_CHAR_RANGE = 27;

struct Driver {
    string name;
    string nationality;
    int birthYear;
    string status;
};

struct User {
    string username;
    string password;
};

struct Team {
    string name;
};

struct Contract {
    int driverIndex;
    int teamIndex;
    int startYear;
    int endYear;
};

// ===== HELPER =====

void clearInput() {
    cin.clear();
}
```

```

        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    bool isValidDriverIndex(int index, int totalDrivers) {
        return index >= 0 && index < totalDrivers;
    }

    bool isValidTeamIndex(int index, int totalTeams) {
        return index >= 0 && index < totalTeams;
    }

    bool isContractActiveInYear(const Contract &contract, int year) {
        return contract.startYear <= year && contract.endYear >= year;
    }

    int getAge(const Driver &driver) {
        return CURRENT_YEAR - driver.birthYear;
    }

    string toUpperCase(string text) {
        for (string::size_type i = 0; i < text.length(); i++) {
            text[i] = static_cast<char>(toupper(static_cast<unsigned
char>(text[i])));
        }
        return text;
    }

    int getRadixCharValueFromLeft(const string &text, int posFromLeft) {
        int length = static_cast<int>(text.length());

        if (posFromLeft < 0 || posFromLeft >= length) {
            return 0;
        }

        unsigned char c = static_cast<unsigned char>(
            text[static_cast<string::size_type>(posFromLeft)]
        );

        c = static_cast<unsigned char>(toupper(c));

        if (c >= 'A' && c <= 'Z') {
            return static_cast<int>(c - 'A' + 1);
        }

        return 0;
    }

    int getMaxNameLengthDrivers(Driver drivers[], int totalDrivers) {

```

```

    int maxLen = 0;
    for (int i = 0; i < totalDrivers; i++) {
        int len = static_cast<int>(drivers[i].name.length());
        if (len > maxLen) {
            maxLen = len;
        }
    }
    return maxLen;
}

int getMaxNameLengthContractsByDriver(Contract contracts[], int
totalContracts, Driver drivers[]) {
    int maxLen = 0;
    for (int i = 0; i < totalContracts; i++) {
        int len =
static_cast<int>(drivers[contracts[i].driverIndex].name.length());
        if (len > maxLen) {
            maxLen = len;
        }
    }
    return maxLen;
}

int getMaxNameLengthContractsByTeam(Contract contracts[], int
totalContracts, Team teams[]) {
    int maxLen = 0;
    for (int i = 0; i < totalContracts; i++) {
        int len =
static_cast<int>(teams[contracts[i].teamIndex].name.length());
        if (len > maxLen) {
            maxLen = len;
        }
    }
    return maxLen;
}

// ===== QUICK SORT (ASCENDING) =====

int partitionNationalityAsc(Driver drivers[], int low, int high) {
    string pivot = drivers[high].nationality;
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (drivers[j].nationality < pivot) {
            i++;
            swap(drivers[i], drivers[j]);
        }
    }
}

```

```

        swap(drivers[i + 1], drivers[high]);
        return i + 1;
    }

void quickSortByNationalityAsc(Driver drivers[], int low, int high) {
    if (low < high) {
        int pivot = partitionNationalityAsc(drivers, low, high);
        quickSortByNationalityAsc(drivers, low, pivot - 1);
        quickSortByNationalityAsc(drivers, pivot + 1, high);
    }
}

int partitionBirthYearAsc(Driver drivers[], int low, int high) {
    int pivot = drivers[high].birthYear;
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (drivers[j].birthYear < pivot) {
            i++;
            swap(drivers[i], drivers[j]);
        }
    }

    swap(drivers[i + 1], drivers[high]);
    return i + 1;
}

void quickSortByBirthYearAsc(Driver drivers[], int low, int high) {
    if (low < high) {
        int pivot = partitionBirthYearAsc(drivers, low, high);
        quickSortByBirthYearAsc(drivers, low, pivot - 1);
        quickSortByBirthYearAsc(drivers, pivot + 1, high);
    }
}

int partitionAgeAsc(Driver drivers[], int low, int high) {
    int pivot = getAge(drivers[high]);
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (getAge(drivers[j]) < pivot) {
            i++;
            swap(drivers[i], drivers[j]);
        }
    }

    swap(drivers[i + 1], drivers[high]);
}

```

```

        return i + 1;
    }

    void quickSortByAgeAsc(Driver drivers[], int low, int high) {
        if (low < high) {
            int pivot = partitionAgeAsc(drivers, low, high);
            quickSortByAgeAsc(drivers, low, pivot - 1);
            quickSortByAgeAsc(drivers, pivot + 1, high);
        }
    }

    int partitionContractStartYearAsc(Contract contracts[], int low, int high) {
        int pivot = contracts[high].startYear;
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (contracts[j].startYear < pivot) {
                i++;
                swap(contracts[i], contracts[j]);
            }
        }

        swap(contracts[i + 1], contracts[high]);
        return i + 1;
    }

    void quickSortContractByStartYearAsc(Contract contracts[], int low, int high) {
        if (low < high) {
            int pivot = partitionContractStartYearAsc(contracts, low, high);
            quickSortContractByStartYearAsc(contracts, low, pivot - 1);
            quickSortContractByStartYearAsc(contracts, pivot + 1, high);
        }
    }

    int partitionContractEndYearAsc(Contract contracts[], int low, int high) {
        int pivot = contracts[high].endYear;
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (contracts[j].endYear < pivot) {
                i++;
                swap(contracts[i], contracts[j]);
            }
        }

        swap(contracts[i + 1], contracts[high]);
        return i + 1;
    }

```

```

}

void quickSortContractByEndYearAsc(Contract contracts[], int low, int high)
{
    if (low < high) {
        int pivot = partitionContractEndYearAsc(contracts, low, high);
        quickSortContractByEndYearAsc(contracts, low, pivot - 1);
        quickSortContractByEndYearAsc(contracts, pivot + 1, high);
    }
}

// ===== BUBBLE SORT (DESCENDING) =====

void bubbleSortByAgeDesc(Driver drivers[], int totalDrivers) {
    for (int i = 0; i < totalDrivers - 1; i++) {
        for (int j = 0; j < totalDrivers - i - 1; j++) {
            if (getAge(drivers[j]) < getAge(drivers[j + 1])) {
                swap(drivers[j], drivers[j + 1]);
            }
        }
    }
}

void bubbleSortByBirthYearDesc(Driver drivers[], int totalDrivers) {
    for (int i = 0; i < totalDrivers - 1; i++) {
        for (int j = 0; j < totalDrivers - i - 1; j++) {
            if (drivers[j].birthYear < drivers[j + 1].birthYear) {
                swap(drivers[j], drivers[j + 1]);
            }
        }
    }
}

void bubbleSortByContractStartYearDesc(Contract contracts[], int
totalContracts) {
    for (int i = 0; i < totalContracts - 1; i++) {
        for (int j = 0; j < totalContracts - i - 1; j++) {
            if (contracts[j].startYear < contracts[j + 1].startYear) {
                swap(contracts[j], contracts[j + 1]);
            }
        }
    }
}

void bubbleSortByContractEndYearDesc(Contract contracts[], int
totalContracts) {
    for (int i = 0; i < totalContracts - 1; i++) {
        for (int j = 0; j < totalContracts - i - 1; j++) {

```



```

        if (contracts[j].endYear < contracts[j + 1].endYear) {
            swap(contracts[j], contracts[j + 1]);
        }
    }
}

// ===== RADIX SORT NAMA =====

void countingSortDriversByName(Driver drivers[], int totalDrivers, int
posFromLeft) {
    Driver output[MAX_DRIVERS];
    int count[RADIX_CHAR_RANGE] = {0};

    for (int i = 0; i < totalDrivers; i++) {
        int key = getRadixCharValueFromLeft(drivers[i].name, posFromLeft);
        count[key]++;
    }

    for (int i = 1; i < RADIX_CHAR_RANGE; i++) {
        count[i] += count[i - 1];
    }

    for (int i = totalDrivers - 1; i >= 0; i--) {
        int key = getRadixCharValueFromLeft(drivers[i].name, posFromLeft);
        output[count[key] - 1] = drivers[i];
        count[key]--;
    }

    for (int i = 0; i < totalDrivers; i++) {
        drivers[i] = output[i];
    }
}

void radixSortDriversByName(Driver drivers[], int totalDrivers) {
    int maxLen = getMaxNameLengthDrivers(drivers, totalDrivers);

    for (int pos = maxLen - 1; pos >= 0; pos--) {
        countingSortDriversByName(drivers, totalDrivers, pos);
    }
}

void countingSortContractsByDriverName(Contract contracts[], int
totalContracts, Driver drivers[], int posFromLeft) {
    Contract output[MAX_CONTRACTS];
    int count[RADIX_CHAR_RANGE] = {0};

    for (int i = 0; i < totalContracts; i++) {

```

```

        int key =
getRadixCharValueFromLeft(drivers[contracts[i].driverIndex].name,
posFromLeft);
        count[key]++;
    }

    for (int i = 1; i < RADIX_CHAR_RANGE; i++) {
        count[i] += count[i - 1];
    }

    for (int i = totalContracts - 1; i >= 0; i--) {
        int key =
getRadixCharValueFromLeft(drivers[contracts[i].driverIndex].name,
posFromLeft);
        output[count[key] - 1] = contracts[i];
        count[key]--;
    }

    for (int i = 0; i < totalContracts; i++) {
        contracts[i] = output[i];
    }
}

void radixSortContractsByDriverName(Contract contracts[], int
totalContracts, Driver drivers[]) {
    int maxLen = getMaxNameLengthContractsByDriver(contracts,
totalContracts, drivers);

    for (int pos = maxLen - 1; pos >= 0; pos--) {
        countingSortContractsByDriverName(contracts, totalContracts,
drivers, pos);
    }
}

void countingSortContractsByTeamName(Contract contracts[], int
totalContracts, Team teams[], int posFromLeft) {
    Contract output[MAX_CONTRACTS];
    int count[RADIX_CHAR_RANGE] = {0};

    for (int i = 0; i < totalContracts; i++) {
        int key =
getRadixCharValueFromLeft(teams[contracts[i].teamIndex].name, posFromLeft);
        count[key]++;
    }

    for (int i = 1; i < RADIX_CHAR_RANGE; i++) {
        count[i] += count[i - 1];
    }
}

```

```

        for (int i = totalContracts - 1; i >= 0; i--) {
            int key =
getRadixCharValueFromLeft(teams[contracts[i].teamIndex].name, posFromLeft);
            output[count[key] - 1] = contracts[i];
            count[key]--;
        }

        for (int i = 0; i < totalContracts; i++) {
            contracts[i] = output[i];
        }
    }

void radixSortContractsByTeamName(Contract contracts[], int totalContracts,
Team teams[]) {
    int maxLen = getMaxNameLengthContractsByTeam(contracts, totalContracts,
teams);

    for (int pos = maxLen - 1; pos >= 0; pos--) {
        countingSortContractsByTeamName(contracts, totalContracts, teams,
pos);
    }
}

// ===== INIT =====

void initializeDrivers(Driver drivers[], int &totalDrivers) {
    string names[] = {
        "Charles Leclerc", "Lewis Hamilton", "George Russell", "Kimi
Antonelli",
        "Lando Norris", "Oscar Piastri", "Fernando Alonso", "Lance Stroll",
        "Esteban Ocon", "Oliver Bearman", "Max Verstappen", "Sergio Perez",
        "Arvid Lindblad", "Pierre Gasly", "Valtteri Bottas", "Isack Hadjar",
        "Nico Hulkenberg", "Gabriel Bortoletto", "Liam Lawson", "Alexander
Albon",
        "Carlos Sainz", "Franco Colapinto"
    };

    string countries[] = {
        "Monaco", "UK", "UK", "Italy",
        "UK", "Australia", "Spain", "Canada",
        "France", "UK", "Netherlands", "Mexico",
        "Sweden", "France", "Finland", "France",
        "Germany", "Brazil", "New Zealand", "Thailand",
        "Spain", "Argentina"
    };

    int birthYears[] = {

```

```

        1997, 1985, 1998, 2006,
        1999, 2001, 1981, 1998,
        1996, 2005, 1997, 1990,
        2004, 1996, 1989, 2003,
        1987, 2004, 2002, 1996,
        1994, 2003
    };

    for (int i = 0; i < 22; i++) {
        drivers[i].name = names[i];
        drivers[i].nationality = countries[i];
        drivers[i].birthYear = birthYears[i];
        drivers[i].status = "Aktif";
    }

    totalDrivers = 22;
}

void initializeTeams(Team teams[], int &totalTeams) {
    string names[] = {
        "Ferrari", "Mercedes", "McLaren", "Red Bull", "RB",
        "Alpine", "Aston Martin", "Audi", "Williams", "Cadillac", "Haas"
    };

    for (int i = 0; i < 11; i++) {
        teams[i].name = names[i];
    }

    totalTeams = 11;
}

void initializeUsers(User users[], int &totalUsers) {
    users[0].username = "Atto1";
    users[0].password = "2509106103";
    totalUsers = 1;
}

void initializeContracts(Contract contracts[], int &totalContracts) {
    int map[22] = {0, 0, 1, 1, 2, 2, 6, 6, 10, 10, 3, 9, 4, 5, 9, 3, 7, 7,
4, 8, 8, 5};

    for (int i = 0; i < 22; i++) {
        contracts[i].driverIndex = i;
        contracts[i].teamIndex = map[i];
        contracts[i].startYear = 2025;
        contracts[i].endYear = 2027;
    }
}

```

```

        totalContracts = 22;
    }

    // ===== USER =====

    void registerUser(User users[], int &totalUsers) {
        if (totalUsers >= MAX_USERS) {
            cout << "Kapasitas user penuh!\n";
            return;
        }

        cout << "Username: ";
        getline(cin, users[totalUsers].username);

        cout << "Password: ";
        getline(cin, users[totalUsers].password);

        totalUsers++;
        cout << "Registrasi berhasil!\n";
    }

    bool loginUser(User users[], int totalUsers) {
        string username, password;

        cout << "Username: ";
        getline(cin, username);

        cout << "Password: ";
        getline(cin, password);

        for (int i = 0; i < totalUsers; i++) {
            if (users[i].username == username && users[i].password == password)
            {
                return true;
            }
        }

        cout << "Login gagal!\n";
        return false;
    }

    // ===== DISPLAY =====

    void displayDrivers(Driver drivers[], int totalDrivers) {
        cout << "\n=== LIST DRIVER ===\n";
        cout << left
            << setw(4) << "No"
            << setw(25) << "Nama"

```

```

        << setw(15) << "Negara"
        << setw(12) << "Tahun Lahir"
        << setw(8) << "Umur"
        << setw(15) << "Status" << '\n';

    cout << string(79, '=') << '\n';

    for (int i = 0; i < totalDrivers; i++) {
        cout << left
            << setw(4) << i + 1
            << setw(25) << drivers[i].name
            << setw(15) << drivers[i].nationality
            << setw(12) << drivers[i].birthYear
            << setw(8) << getAge(drivers[i])
            << setw(15) << drivers[i].status
            << '\n';
    }
}

void displayTeams(Team teams[], int totalTeams) {
    cout << "\n=== LIST TEAM ===\n";
    for (int i = 0; i < totalTeams; i++) {
        cout << i + 1 << ". " << teams[i].name << '\n';
    }
}

void displayAllContracts(Contract contracts[], int totalContracts, Driver
drivers[], Team teams[]) {
    cout << "\n=== LIST SEMUA KONTRAK ===\n";
    cout << left
        << setw(4) << "No"
        << setw(25) << "Driver"
        << setw(20) << "Tim"
        << setw(12) << "Mulai"
        << setw(12) << "Selesai"
        << setw(15) << "Status 2026"
        << '\n';

    cout << string(88, '=') << '\n';

    for (int i = 0; i < totalContracts; i++) {
        string status2026 = isContractActiveInYear(contracts[i],
CURRENT_YEAR) ? "Aktif" : "Tidak Aktif";

        cout << left
            << setw(4) << i + 1
            << setw(25) << drivers[contracts[i].driverIndex].name
            << setw(20) << teams[contracts[i].teamIndex].name

```

```

        << setw(12) << contracts[i].startYear
        << setw(12) << contracts[i].endYear
        << setw(15) << status2026
        << '\n';
    }

    if (totalContracts == 0) {
        cout << "Belum ada data kontrak.\n";
    }
}

void displayActiveContracts2026(Contract contracts[], int totalContracts,
Driver drivers[], Team teams[]) {
    cout << "\n=== LIST KONTRAK AKTIF TAHUN " << CURRENT_YEAR << " ===\n";
    cout << left
        << setw(4) << "No"
        << setw(25) << "Driver"
        << setw(20) << "Tim"
        << setw(12) << "Mulai"
        << setw(12) << "Selesai"
        << '\n';

    cout << string(73, '=') << '\n';

    int nomor = 1;
    for (int i = 0; i < totalContracts; i++) {
        if (isContractActiveInYear(contracts[i], CURRENT_YEAR)) {
            cout << left
                << setw(4) << nomor
                << setw(25) << drivers[contracts[i].driverIndex].name
                << setw(20) << teams[contracts[i].teamIndex].name
                << setw(12) << contracts[i].startYear
                << setw(12) << contracts[i].endYear
                << '\n';
            nomor++;
        }
    }

    if (nomor == 1) {
        cout << "Tidak ada kontrak aktif pada tahun " << CURRENT_YEAR <<
        ".\n";
    }
}

void displayFoundDriver(const Driver &driver, int index) {
    cout << "\nData ditemukan pada index ke-" << index << '\n';
    cout << left
        << setw(25) << "Nama" << ": " << driver.name << '\n'

```

```

        << setw(25) << "Negara" << ": " << driver.nationality << '\n'
        << setw(25) << "Tahun lahir" << ": " << driver.birthYear << '\n'
        << setw(25) << "Umur" << ": " << getAge(driver) << '\n'
        << setw(25) << "Status" << ": " << driver.status << '\n';
    }

```

```

// ===== SEARCH =====

```

```

int jumpSearchByName(Driver *drivers, int totalDrivers, const string
&target) {
    if (totalDrivers <= 0) {
        return -1;
    }

    string targetUpper = toUpperCase(target);

    int step = 1;
    while (step * step < totalDrivers) {
        step++;
    }

    int prev = 0;
    int current = step;

    while (prev < totalDrivers &&
        toUpperCase((drivers + ((current < totalDrivers) ? current :
totalDrivers) - 1)->name) < targetUpper) {
        prev = current;
        current += step;

        if (prev >= totalDrivers) {
            return -1;
        }
    }

    int end = (current < totalDrivers) ? current : totalDrivers;

    for (int i = prev; i < end; i++) {
        if (toUpperCase((drivers + i)->name) == targetUpper) {
            return i;
        }
    }

    return -1;
}

```

```

int binarySearchByAge(Driver *drivers, int totalDrivers, int targetAge) {
    int left = 0;

```



```

    int right = totalDrivers - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int ageMid = getAge(*(drivers + mid));

        if (ageMid == targetAge) {
            return mid;
        } else if (ageMid < targetAge) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}

void linearSearchByBirthYear(Driver *drivers, int totalDrivers, int
targetBirthYear) {
    bool found = false;

    cout << "\n=== HASIL PENCARIAN TAHUN LAHIR ===\n";
    for (int i = 0; i < totalDrivers; i++) {
        if ((drivers + i)->birthYear == targetBirthYear) {
            found = true;
            displayFoundDriver(*(drivers + i), i);
        }
    }

    if (!found) {
        cout << "Data dengan tahun lahir " << targetBirthYear << " tidak
ditemukan.\n";
    }
}

void searchDriverMenu(Driver drivers[], int totalDrivers) {
    int choice;

    cout << "\n=== SEARCH DRIVER ===\n";
    cout << "1. Nama Driver (Jump Search)\n";
    cout << "2. Umur (Binary Search)\n";
    cout << "3. Tahun Lahir (Linear Search)\n";
    cout << "Pilih: ";
    cin >> choice;

    if (cin.fail()) {
        clearInput();
    }
}

```

```

        cout << "Input tidak valid!\n";
        return;
    }

    if (choice == 1) {
        clearInput();
        string targetName;
        cout << "Masukkan nama driver: ";
        getline(cin, targetName);

        Driver temp[MAX_DRIVERS];
        for (int i = 0; i < totalDrivers; i++) {
            temp[i] = drivers[i];
        }

        radixSortDriversByName(temp, totalDrivers);

        int index = jumpSearchByName(temp, totalDrivers, targetName);
        if (index != -1) {
            displayFoundDriver(temp[index], index);
        } else {
            cout << "Driver dengan nama \"" << targetName << "\" tidak
ditemukan.\n";
        }
    } else if (choice == 2) {
        int targetAge;
        cout << "Masukkan umur: ";
        cin >> targetAge;

        if (cin.fail()) {
            clearInput();
            cout << "Input umur tidak valid!\n";
            return;
        }

        Driver temp[MAX_DRIVERS];
        for (int i = 0; i < totalDrivers; i++) {
            temp[i] = drivers[i];
        }

        quickSortByAgeAsc(temp, 0, totalDrivers - 1);

        int index = binarySearchByAge(temp, totalDrivers, targetAge);
        if (index != -1) {
            displayFoundDriver(temp[index], index);
        } else {
            cout << "Driver dengan umur " << targetAge << " tidak
ditemukan.\n";
        }
    }
}

```

```

    }
} else if (choice == 3) {
    int targetBirthYear;
    cout << "Masukkan tahun lahir: ";
    cin >> targetBirthYear;

    if (cin.fail()) {
        clearInput();
        cout << "Input tahun lahir tidak valid!\n";
        return;
    }

    linearSearchByBirthYear(drivers, totalDrivers, targetBirthYear);
} else {
    cout << "Pilihan search tidak valid!\n";
}
}

// ===== SORT MENU DRIVER =====

void sortDriversMenu(Driver drivers[], int totalDrivers) {
    int sortChoice;

    cout << "\n=== SORT DRIVER BY ===\n";
    cout << "1. Umur Descending (Bubble Sort)\n";
    cout << "2. Negara Ascending (Quick Sort)\n";
    cout << "3. Tahun Lahir Descending (Bubble Sort)\n";
    cout << "4. Nama Ascending (Radix Sort)\n";
    cout << "Pilih: ";
    cin >> sortChoice;

    if (cin.fail()) {
        clearInput();
        cout << "Input tidak valid!\n";
        return;
    }

    switch (sortChoice) {
        case 1:
            bubbleSortByAgeDesc(drivers, totalDrivers);
            break;
        case 2:
            quickSortByNationalityAsc(drivers, 0, totalDrivers - 1);
            break;
        case 3:
            bubbleSortByBirthYearDesc(drivers, totalDrivers);
            break;
        case 4:

```

```

        radixSortDriversByName(drivers, totalDrivers);
        break;
    default:
        cout << "Pilihan sort tidak valid!\n";
        return;
    }

    displayDrivers(drivers, totalDrivers);
}

// ===== SORT MENU KONTRAK =====

void sortContractsMenu(Contract contracts[], int totalContracts, Driver
drivers[], Team teams[]) {
    int sortChoice;
    int filterChoice;

    cout << "\n=== SORT KONTRAK BY ===\n";
    cout << "1. Nama Driver Ascending (Radix Sort)\n";
    cout << "2. Nama Tim Ascending (Radix Sort)\n";
    cout << "3. Tahun Mulai Descending (Bubble Sort)\n";
    cout << "4. Tahun Selesai Descending (Bubble Sort)\n";
    cout << "Pilih: ";
    cin >> sortChoice;

    if (cin.fail()) {
        clearInput();
        cout << "Input tidak valid!\n";
        return;
    }

    switch (sortChoice) {
        case 1:
            radixSortContractsByDriverName(contracts, totalContracts,
drivers);
            break;
        case 2:
            radixSortContractsByTeamName(contracts, totalContracts, teams);
            break;
        case 3:
            bubbleSortByContractStartYearDesc(contracts, totalContracts);
            break;
        case 4:
            bubbleSortByContractEndYearDesc(contracts, totalContracts);
            break;
        default:
            cout << "Pilihan sort tidak valid!\n";
            return;
    }
}

```

```

    }

    cout << "\n=== TAMPILKAN KONTRAK ===\n";
    cout << "1. Semua kontrak\n";
    cout << "2. Hanya kontrak aktif tahun 2026\n";
    cout << "Pilih: ";
    cin >> filterChoice;

    if (cin.fail()) {
        clearInput();
        cout << "Input tidak valid!\n";
        return;
    }

    switch (filterChoice) {
        case 1:
            displayAllContracts(contracts, totalContracts, drivers, teams);
            break;
        case 2:
            displayActiveContracts2026(contracts, totalContracts, drivers,
teams);
            break;
        default:
            cout << "Pilihan tampilan tidak valid!\n";
            break;
    }
}

// ===== CRUD DRIVER =====

void addDriver(Driver drivers[], int &totalDrivers) {
    if (totalDrivers >= MAX_DRIVERS) {
        cout << "Data driver penuh!\n";
        return;
    }

    clearInput();

    cout << "Nama: ";
    getline(cin, drivers[totalDrivers].name);

    cout << "Negara: ";
    getline(cin, drivers[totalDrivers].nationality);

    cout << "Tahun lahir: ";
    cin >> drivers[totalDrivers].birthYear;

    if (cin.fail()) {

```

```

        clearInput();
        cout << "Input tahun lahir tidak valid!\n";
        return;
    }

    clearInput();

    cout << "Status: ";
    getline(cin, drivers[totalDrivers].status);

    totalDrivers++;
    cout << "Driver berhasil ditambahkan!\n";
}

void editDriver(Driver drivers[], int totalDrivers) {
    if (totalDrivers == 0) {
        cout << "Tidak ada data driver.\n";
        return;
    }

    displayDrivers(drivers, totalDrivers);

    int choice;
    cout << "Pilih nomor driver yang ingin diedit: ";
    cin >> choice;

    if (cin.fail()) {
        clearInput();
        cout << "Input tidak valid!\n";
        return;
    }

    int index = choice - 1;
    if (!isValidDriverIndex(index, totalDrivers)) {
        cout << "Nomor driver tidak valid!\n";
        return;
    }

    clearInput();

    cout << "Nama baru: ";
    getline(cin, drivers[index].name);

    cout << "Negara baru: ";
    getline(cin, drivers[index].nationality);

    cout << "Tahun lahir baru: ";
    cin >> drivers[index].birthYear;
}

```

```

        if (cin.fail()) {
            clearInput();
            cout << "Input tahun lahir tidak valid!\n";
            return;
        }

        clearInput();

        cout << "Status baru: ";
        getline(cin, drivers[index].status);

        cout << "Data driver berhasil diupdate!\n";
    }

void deleteDriverContracts(Contract contracts[], int &totalContracts, int
driverIndex) {
    for (int i = 0; i < totalContracts; i++) {
        if (contracts[i].driverIndex == driverIndex) {
            for (int j = i; j < totalContracts - 1; j++) {
                contracts[j] = contracts[j + 1];
            }
            totalContracts--;
            i--;
        }
    }
}

void adjustContractDriverIndexes(Contract contracts[], int totalContracts,
int deletedDriverIndex) {
    for (int i = 0; i < totalContracts; i++) {
        if (contracts[i].driverIndex > deletedDriverIndex) {
            contracts[i].driverIndex--;
        }
    }
}

void deleteDriver(Driver drivers[], int &totalDrivers, Contract contracts[],
int &totalContracts) {
    if (totalDrivers == 0) {
        cout << "Tidak ada data driver.\n";
        return;
    }

    displayDrivers(drivers, totalDrivers);

    int choice;
    cout << "Pilih nomor driver yang ingin dihapus: ";

```

```

    cin >> choice;

    if (cin.fail()) {
        clearInput();
        cout << "Input tidak valid!\n";
        return;
    }

    int index = choice - 1;
    if (!isValidDriverIndex(index, totalDrivers)) {
        cout << "Nomor driver tidak valid!\n";
        return;
    }

    deleteDriverContracts(contracts, totalContracts, index);
    adjustContractDriverIndexes(contracts, totalContracts, index);

    for (int i = index; i < totalDrivers - 1; i++) {
        drivers[i] = drivers[i + 1];
    }

    totalDrivers--;
    cout << "Driver berhasil dihapus!\n";
}

// ===== KONTRAK =====

void addContract(Driver drivers[], int totalDrivers,
                Team teams[], int totalTeams,
                Contract contracts[], int &totalContracts) {
    if (totalContracts >= MAX_CONTRACTS) {
        cout << "Data kontrak penuh!\n";
        return;
    }

    int driverChoice;
    int teamChoice;
    int startYear;
    int endYear;

    displayDrivers(drivers, totalDrivers);
    cout << "Pilih driver: ";
    cin >> driverChoice;

    if (cin.fail()) {
        clearInput();
        cout << "Input driver tidak valid!\n";
        return;
    }

```



```

}

int driverIndex = driverChoice - 1;
if (!isValidDriverIndex(driverIndex, totalDrivers)) {
    cout << "Pilihan driver tidak valid!\n";
    return;
}

displayTeams(teams, totalTeams);
cout << "Pilih tim: ";
cin >> teamChoice;

if (cin.fail()) {
    clearInput();
    cout << "Input tim tidak valid!\n";
    return;
}

int teamIndex = teamChoice - 1;
if (!isValidTeamIndex(teamIndex, totalTeams)) {
    cout << "Pilihan tim tidak valid!\n";
    return;
}

cout << "Tahun mulai: ";
cin >> startYear;
if (cin.fail()) {
    clearInput();
    cout << "Input tahun mulai tidak valid!\n";
    return;
}

cout << "Tahun selesai: ";
cin >> endYear;
if (cin.fail()) {
    clearInput();
    cout << "Input tahun selesai tidak valid!\n";
    return;
}

if (endYear < startYear) {
    cout << "Tahun selesai tidak boleh lebih kecil dari tahun mulai!\n";
    return;
}

contracts[totalContracts].driverIndex = driverIndex;
contracts[totalContracts].teamIndex = teamIndex;
contracts[totalContracts].startYear = startYear;

```

```

        contracts[totalContracts].endYear = endYear;
        totalContracts++;

        drivers[driverIndex].status = "Aktif";

        cout << "Kontrak berhasil ditambahkan!\n";
    }

void transferDriver(Driver drivers[], int totalDrivers,
                    Team teams[], int totalTeams,
                    Contract contracts[], int &totalContracts) {
    addContract(drivers, totalDrivers, teams, totalTeams, contracts,
totalContracts);
}

// ===== MENU ADMIN =====

void adminMenu(Driver drivers[], int &totalDrivers,
                Team teams[], int totalTeams,
                Contract contracts[], int &totalContracts) {
    int choice;

    do {
        cout << "\n=== MENU ADMIN ===\n";
        cout << "1. List Driver\n";
        cout << "2. Search Driver\n";
        cout << "3. Tambah Driver\n";
        cout << "4. Edit Driver\n";
        cout << "5. Hapus Driver\n";
        cout << "6. List Kontrak\n";
        cout << "7. Tambah / Transfer Kontrak\n";
        cout << "8. Logout\n";
        cout << "Pilih: ";
        cin >> choice;

        if (cin.fail()) {
            clearInput();
            cout << "Input tidak valid!\n";
            continue;
        }

        switch (choice) {
            case 1:
                sortDriversMenu(drivers, totalDrivers);
                break;
            case 2:
                searchDriverMenu(drivers, totalDrivers);
                break;

```

```

        case 3:
            addDriver(drivers, totalDrivers);
            break;
        case 4:
            editDriver(drivers, totalDrivers);
            break;
        case 5:
            deleteDriver(drivers, totalDrivers, contracts,
totalContracts);
            break;
        case 6:
            if (totalContracts > 0) {
                sortContractsMenu(contracts, totalContracts, drivers,
teams);
            } else {
                cout << "Tidak ada data kontrak.\n";
            }
            break;
        case 7:
            transferDriver(drivers, totalDrivers, teams, totalTeams,
contracts, totalContracts);
            break;
        case 8:
            cout << "Logout...\n";
            break;
        default:
            cout << "Menu belum tersedia.\n";
    }

    } while (choice != 8);
}

// ===== MAIN =====

int main() {
    Driver drivers[MAX_DRIVERS];
    User users[MAX_USERS];
    Team teams[MAX_TEAMS];
    Contract contracts[MAX_CONTRACTS];

    int totalDrivers = 0;
    int totalUsers = 0;
    int totalTeams = 0;
    int totalContracts = 0;

    initializeDrivers(drivers, totalDrivers);
    initializeTeams(teams, totalTeams);
    initializeUsers(users, totalUsers);

```

```

        initializeContracts(contracts, totalContracts);

        int choice;

        do {
            cout << "\n=== MENU AWAL ===\n";
            cout << "1. Register\n";
            cout << "2. Login\n";
            cout << "3. Keluar\n";
            cout << "Pilih: ";
            cin >> choice;

            if (cin.fail()) {
                clearInput();
                cout << "Input tidak valid!\n";
                continue;
            }

            clearInput();

            if (choice == 1) {
                registerUser(users, totalUsers);
            } else if (choice == 2) {
                if (loginUser(users, totalUsers)) {
                    adminMenu(drivers, totalDrivers, teams, totalTeams,
contracts, totalContracts);
                }
            } else if (choice == 3) {
                cout << "Program selesai.\n";
            } else {
                cout << "Pilihan tidak valid.\n";
            }

        } while (choice != 3);

        return 0;
    }
}

```

4. Hasil Output

```
+-----+
|      SELAMAT DATANG DI BURSA TRANSFER F1      |
+-----+

=====
MENU AWAL
=====
+-----+
| 1. Register Admin      |
| 2. Login Admin         |
| 3. Keluar              |
+-----+
Pilihan : 2
Username : Attol
Password : 2509106103
Login berhasil.
```

Gambar 4.1 Hasil Output Menu Awal

```
=== MENU ===
1. List Driver
2. Tambah Driver
3. Edit Driver
4. Hapus Driver
5. List Kontrak
6. Transfer Driver
7. Logout
Pilih: █
```

Gambar 4.2 Hasil Output Login

```
=====
```

DATA PEMBALAP LENGKAP						
=====						
No	Nama	Negara	Tahun Lahir	Umur	Status	

1	Charles Leclerc	Monaco	1997	29	Aktif	
2	Lewis Hamilton	UK	1985	41	Aktif	
3	George Russell	UK	1998	28	Aktif	
4	Kimi Antonelli	Italy	2006	20	Aktif	
5	Lando Norris	UK	1999	27	Aktif	
6	Oscar Piastri	Australia	2001	25	Aktif	
7	Fernando Alonso	Spain	1981	45	Aktif	
8	Lance Stroll	Canada	1998	28	Aktif	
9	Esteban Ocon	France	1996	30	Aktif	
10	Oliver Bearman	UK	2005	21	Aktif	
11	Isack Hadjar	France	2003	23	Aktif	
12	Max Verstappen	Netherlands	1997	29	Aktif	
13	Carlos Sainz Jr.	Spain	1994	32	Aktif	
14	Alexander Albon	UK	1996	30	Aktif	
15	Valtteri Bottas	Finland	1989	37	Aktif	
16	Sergio Perez	Mexico	1990	36	Aktif	
17	Nico Hulkenberg	Germany	1987	39	Aktif	
18	Gabriel Bortoleto	Brazil	2004	22	Aktif	
19	Pierre Gasly	France	1996	30	Aktif	
20	Franco Colapinto	Argentina	1999	27	Aktif	
21	Liam Lawson	New Zealand	2002	24	Aktif	
22	Arvid Lindblad	Sweden	2004	22	Aktif	

Gambar 4.3 Hasil Output List pembalap

```
=== MENU ===
1. List Driver
2. Tambah Driver
3. Edit Driver
4. Hapus Driver
5. List Kontrak
6. Transfer Driver
7. Logout
Pilih: 2
Nama: Benjamin Seskowi
Negara: Solovenia
Tahun lahir: 2002
Status (Aktif/Tidak Aktif/Free Agent): Aktif
Driver berhasil ditambahkan!
```

Gambar 4.4 Output Create Pembalap

```

=== LIST DRIVER ===

```

No	Nama	Negara	Tahun Lahir	Umur	Status
=====					
1	Fernando Alonso	Spain	1981	45	Aktif
2	Lewis Hamilton	UK	1985	41	Aktif
3	Nico Hulkenberg	Germany	1987	39	Aktif
4	Valtteri Bottas	Finland	1989	37	Aktif
5	Sergio Perez	Mexico	1990	36	Aktif
6	Carlos Sainz	Spain	1994	32	Aktif
7	Alexander Albon	Thailand	1996	30	Aktif
8	Esteban Ocon	France	1996	30	Aktif
9	Pierre Gasly	France	1996	30	Aktif
10	Charles Leclerc	Monaco	1997	29	Aktif
11	Max Verstappen	Netherlands	1997	29	Aktif
12	George Russell	UK	1998	28	Aktif
13	Lance Stroll	Canada	1998	28	Aktif
14	Lando Norris	UK	1999	27	Aktif
15	Oscar Piastri	Australia	2001	25	Aktif
16	Liam Lawson	New Zealand	2002	24	Aktif
17	Franco Colapinto	Argentina	2003	23	Aktif
18	Isack Hadjar	France	2003	23	Aktif
19	Arvid Lindblad	Sweden	2004	22	Aktif
20	Gabriel Bortoletto	Brazil	2004	22	Aktif
21	Oliver Bearman	UK	2005	21	Aktif
22	Kimi Antonelli	Italy	2006	20	Aktif

Gambar 4.5 CRUD list pembalap sort berdasarkan umur(Descending) (Bubble Sort)

```

=== LIST DRIVER ===

```

No	Nama	Negara	Tahun Lahir	Umur	Status
=====					
1	Franco Colapinto	Argentina	2003	23	Aktif
2	Oscar Piastri	Australia	2001	25	Aktif
3	Gabriel Bortoletto	Brazil	2004	22	Aktif
4	Lance Stroll	Canada	1998	28	Aktif
5	Valtteri Bottas	Finland	1989	37	Aktif
6	Pierre Gasly	France	1996	30	Aktif
7	Esteban Ocon	France	1996	30	Aktif
8	Isack Hadjar	France	2003	23	Aktif
9	Nico Hulkenberg	Germany	1987	39	Aktif
10	Kimi Antonelli	Italy	2006	20	Aktif
11	Sergio Perez	Mexico	1990	36	Aktif
12	Charles Leclerc	Monaco	1997	29	Aktif
13	Max Verstappen	Netherlands	1997	29	Aktif
14	Liam Lawson	New Zealand	2002	24	Aktif
15	Fernando Alonso	Spain	1981	45	Aktif
16	Carlos Sainz	Spain	1994	32	Aktif
17	Arvid Lindblad	Sweden	2004	22	Aktif
18	Alexander Albon	Thailand	1996	30	Aktif
19	George Russell	UK	1998	28	Aktif
20	Lewis Hamilton	UK	1985	41	Aktif
21	Lando Norris	UK	1999	27	Aktif
22	Oliver Bearman	UK	2005	21	Aktif

Gambar 4.6 List sort berdasarkan Negara (Quick Sort)

```

=== LIST DRIVER ===
No  Nama                      Negara      Tahun Lahir  Umur  Status
=====
1   Kimi Antonelli             Italy       2006        20   Aktif
2   Oliver Bearman             UK          2005        21   Aktif
3   Arvid Lindblad             Sweden     2004        22   Aktif
4   Gabriel Bortoletto         Brazil     2004        22   Aktif
5   Isack Hadjar               France     2003        23   Aktif
6   Franco Colapinto           Argentina  2003        23   Aktif
7   Liam Lawson                New Zealand 2002        24   Aktif
8   Oscar Piastri              Australia  2001        25   Aktif
9   Lando Norris               UK         1999        27   Aktif
10  Lance Stroll               Canada     1998        28   Aktif
11  George Russell             UK         1998        28   Aktif
12  Max Verstappen             Netherlands 1997        29   Aktif
13  Charles Leclerc            Monaco     1997        29   Aktif
14  Alexander Albon            Thailand   1996        30   Aktif
15  Pierre Gasly               France     1996        30   Aktif
16  Esteban Ocon               France     1996        30   Aktif
17  Carlos Sainz               Spain      1994        32   Aktif
18  Sergio Perez               Mexico     1990        36   Aktif
19  Valtteri Bottas            Finland    1989        37   Aktif
20  Nico Hulkenberg            Germany    1987        39   Aktif
21  Lewis Hamilton             UK         1985        41   Aktif
22  Fernando Alonso            Spain      1981        45   Aktif

```

Gambar 4.7 Output sort berdasarkan tahun lahir (Bubble sort)

=== LIST DRIVER ===					
No	Nama	Negara	Tahun Lahir	Umur	Status
1	Alexander Albon	Thailand	1996	30	Aktif
2	Arvid Lindblad	Sweden	2004	22	Aktif
3	Carlos Sainz	Spain	1994	32	Aktif
4	Charles Leclerc	Monaco	1997	29	Aktif
5	Esteban Ocon	France	1996	30	Aktif
6	Fernando Alonso	Spain	1981	45	Aktif
7	Franco Colapinto	Argentina	2003	23	Aktif
8	Gabriel Bortoletto	Brazil	2004	22	Aktif
9	George Russell	UK	1998	28	Aktif
10	Isack Hadjar	France	2003	23	Aktif
11	Kimi Antonelli	Italy	2006	20	Aktif
12	Lance Stroll	Canada	1998	28	Aktif
13	Lando Norris	UK	1999	27	Aktif
14	Lewis Hamilton	UK	1985	41	Aktif
15	Liam Lawson	New Zealand	2002	24	Aktif
16	Max Verstappen	Netherlands	1997	29	Aktif
17	Nico Hulkenberg	Germany	1987	39	Aktif
18	Oliver Bearman	UK	2005	21	Aktif
19	Oscar Piastri	Australia	2001	25	Aktif
20	Pierre Gasly	France	1996	30	Aktif
21	Sergio Perez	Mexico	1990	36	Aktif
22	Valtteri Bottas	Finland	1989	37	Aktif

4.8 List Sort berdasarkan Urutan nama (A-Z) (Radix Sort)

```
=====
DATA PEMBALAP LENGKAP
=====
+-----+-----+-----+-----+-----+-----+
| No | Nama | Negara | Tahun Lahir | Umur | Status |
+-----+-----+-----+-----+-----+-----+
| 1 | Charles Leclerc | Monaco | 1997 | 29 | Aktif |
| 2 | Lewis Hamilton | UK | 1985 | 41 | Aktif |
| 3 | George Russell | UK | 1998 | 28 | Aktif |
| 4 | Kimi Antonelli | Italy | 2006 | 20 | Aktif |
| 5 | Lando Norris | UK | 1999 | 27 | Aktif |
| 6 | Oscar Piastri | Australia | 2001 | 25 | Aktif |
| 7 | Fernando Alonso | Spain | 1981 | 45 | Aktif |
| 8 | Lance Stroll | Canada | 1998 | 28 | Aktif |
| 9 | Esteban Ocon | France | 1996 | 30 | Aktif |
| 10 | Oliver Bearman | UK | 2005 | 21 | Aktif |
| 11 | Isack Hadjar | France | 2003 | 23 | Aktif |
| 12 | Max Verstappen | Netherlands | 1997 | 29 | Aktif |
| 13 | Carlos Sainz Jr. | Spain | 1994 | 32 | Aktif |
| 14 | Alexander Albon | UK | 1996 | 30 | Aktif |
| 15 | Valtteri Bottas | Finland | 1989 | 37 | Aktif |
| 16 | Sergio Perez | Mexico | 1990 | 36 | Aktif |
| 17 | Nico Hulkenberg | Germany | 1987 | 39 | Aktif |
| 18 | Gabriel Bortoleto | Brazil | 2004 | 22 | Aktif |
| 19 | Pierre Gasly | France | 1996 | 30 | Aktif |
| 20 | Franco Colapinto | Argentina | 1999 | 27 | Aktif |
| 21 | Liam Lawson | New Zealand | 2002 | 24 | Aktif |
| 22 | Arvid Lindblad | Sweden | 2004 | 22 | Aktif |
| 23 | Benjamin Seskow | Slovenia | 2002 | 24 | Tidak Aktif |
+-----+-----+-----+-----+-----+-----+
Pilih nomor pembalap yang ingin dihapus : 23
Pembalap berhasil dihapus.
```

4.9 Output Delete Pembalap

```
| 22 | Arvid Lindblad | Sweden | 2004 | 22 | Aktif |
+-----+-----+-----+-----+-----+
Pilih nomor pembalap yang ingin diedit : 22
Nama baru : Arvid Limbad
Negara baru : Swedia
Tahun lahir baru : 2007
Pilih status baru:
1. Aktif
2. Tidak Aktif
Pilihan : 1
Pembalap berhasil diedit.
```

Gambar 4.10 Edit Pembalap

```
1. List Driver
2. Tambah Driver
3. Edit Driver
4. Hapus Driver
5. List Kontrak
6. Tambah / Transfer Kontrak
7. Logout
Pilih: 5
```

```
=== SORT KONTRAK BY ===
1. Nama Driver (A-Z)
2. Nama Tim (A-Z)
3. Tahun Mulai (Descending)
4. Tahun Selesai (Descending)
Pilih: 1
```

```
=== TAMPILKAN KONTRAK ===
1. Semua kontrak
2. Hanya kontrak aktif tahun 2026
Pilih: 1
```

```
=== LIST SEMUA KONTRAK ===
No  Driver              Tim              Mulai      Selesai     Status 2026
=====
1   Alexander Albon     Ferrari         2025       2027       Aktif
2   Arvid Lindblad      Ferrari         2025       2027       Aktif
3   Carlos Sainz        Mercedes        2025       2027       Aktif
4   Charles Leclerc     Mercedes        2025       2027       Aktif
5   Esteban Ocon        McLaren         2025       2027       Aktif
6   Fernando Alonso     McLaren         2025       2027       Aktif
7   Franco Colapinto    Aston Martin   2025       2027       Aktif
8   Gabriel Bortoletto  Aston Martin   2025       2027       Aktif
```

Gambar 4.11 Output List Kontrak

```
=== SEARCH DRIVER ===
1. Nama Driver (Fibonacci Search)
2. Umur (Binary Search)
3. Tahun Lahir (Linear Search)
Pilih: 3
Masukkan tahun lahir: 2004

=== HASIL PENCARIAN TAHUN LAHIR ===

Data ditemukan pada index ke-12
Nama                : Arvid Lindblad
Negara              : Sweden
Tahun lahir        : 2004
Umur                : 22
Status             : Aktif

Data ditemukan pada index ke-17
Nama                : Gabriel Bortoletto
Negara              : Brazil
Tahun lahir        : 2004
Umur                : 22
Status             : Aktif
```

Gambar 4.12 Output Search Berdasarkan Tahun Lahir (Linier Search)

```
=== SEARCH DRIVER ===
1. Nama Driver (Fibonacci Search)
2. Umur (Binary Search)
3. Tahun Lahir (Linear Search)
Pilih: 2
Masukkan umur: 30

Data ditemukan pada index ke-13
Nama                : Esteban Ocon
Negara              : France
Tahun lahir        : 1996
Umur                : 30
Status             : Aktif
```

Gambar 4.13 Output search berdasarkan umur (Binary Search)

```

==== SEARCH DRIVER ====
1. Nama Driver (Fibonacci Search)
2. Umur (Binary Search)
3. Tahun Lahir (Linear Search)
Pilih: 1
Masukkan nama driver: Charles Leclerc

Data ditemukan pada index ke-3
Nama                : Charles Leclerc
Negara              : Monaco
Tahun lahir        : 1997
Umur                : 29
Status              : Aktif

```

Gambar 4.14 Output search berdasarkan Nama (Jump Search)

5. Langkah-langkah GIT

```

PS C:\Users\Asus GK\OneDrive\Documents\praktikum-apd\post-test\post-test-7> git add .
PS C:\Users\Asus GK\OneDrive\Documents\praktikum-apd\post-test\post-test-7> git commit -m "KOMAT KAMIT"
[main aede32e] KOMAT KAMIT
3 files changed, 233 insertions(+)
create mode 100644 post-test/post-test-7/2509106103-KhayzanDwittraAttala-PT-7 (1).pdf
create mode 100644 post-test/post-test-7/2509106103-PT-7.drawio.png
create mode 100644 post-test/post-test-7/2509106103_KhayzanDwittraAttala-PT-7.py
PS C:\Users\Asus GK\OneDrive\Documents\praktikum-apd\post-test\post-test-7> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 868.02 KiB | 27.13 MiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Khayzannn/praktikum-apd.git
 609b85f..aede32e  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Asus GK\OneDrive\Documents\praktikum-apd\post-test\post-test-7>

```

Gambar 5.1 Langkah - Langkah Git

5.1 GIT Add

Git add adalah perintah diGit yang digunakan untuk menambahkan perubahan pada berkas di direktori kerja ke area staging (atau indeks), mempersiapkannya untuk dimasukkan ke dalam commit berikutnya.

5.2 GIT Commit

Git commit adalah perintah di sistem kontrol versi Git untuk menyimpan (melakukan "commit") serangkaian perubahan yang telah dipilih (di-staging) ke dalam repositori lokal Anda, membuat sebuah "snapshot" atau titik pemeriksaan pada riwayat proyek dengan deskripsi perubahan.

5.3 GIT Push

Git push adalah perintah dalam Git yang digunakan untuk mengunggah (mengirimkan) perubahan yang telah dicatat di repositori lokal Anda ke repositori jarak jauh (seperti GitHub), sehingga memperbarui cabang tersebut dan membagikan pekerjaan dengan tim atau menyinkronkan dengan versi pusat.